

최종 발표 시연 가이드

HTML MVP 버튼 하나로 시스템이 스스로 도는 모습을 증명한다

▶ 버튼 실행



파일로 증명



돌발 과제



자기개선

시연 가이드는 다섯 부분으로 구성됩니다

01	시연의 원칙	기본 개념 라이브 · 재생 · 엔진과 화면 분리
02	15 분 진행 순서	화면과 역할 시간표 · MVP 화면 · 버튼 · 역할별 설명
03	돌발 과제 대응	당일 제시 입력 파일 · 합격 기준 · 비교표
04	준비 · 리허설	사흘 전 제출 시연 꾸러미 · 리허설 · 장애 대응
05	평가 · 점검표	최종 발표 10 점 배점 · 감점 · 전날 점검

권장 방식 — HTML MVP 의 버튼으로 시작하고 , 근거는 파일로 보여 줍니다

01

시연의 원칙

시연이 무엇을 증명하는 자리인지 , 어떤 방식으로 실행하는지 먼저 정리한다

시연은 슬라이드가 아니라 돌아가는 시스템을 보여 주는 일입니다

시연 (demo) = 청중 앞에서 시스템을 실제로 실행해 설계서의 주장을 증명하는 것

원칙 1

버튼 하나로 돈다

- ▶ 실행 버튼
- 리서치부터 ⑧까지
- 중간에 손 안 댄

원칙 2

파일로 증명한다

- 단계마다 산출 파일
- 열어서 근거 제시
- 실행 번호로 묶음

원칙 3

사람은 정해진 곳만

- IPS 서명
- 긴급 정지
- 큰 수정 승인

설명은 짧게 , 판단의 근거는 파일에서 보여 줍니다

시연 방식은 라이브 · 재생 · 녹화 세 가지입니다

재생 (replay) 모드 = 저장해 둔 LLM 응답을 다시 읽고, 숫자 계산만 코드로 새로 하는 실행

방식	무엇을 하나	언제 쓰나
라이브 (▶ 버튼)	LLM 을 실제로 호출해 처음부터 실행	필수 — 최소 한 구간
재생 (▷ 버튼)	저장된 응답 + 계산 재실행	시간 부족 · 재현성 확인
녹화 백업	미리 녹화한 실행 영상	API · 네트워크 장애 때만

재생 모드로 다시 돌려 같은 숫자가 나오면 재현성이 증명됩니다

녹화 영상만으로 시연을 대신하면 라이브 실행 점수는 없습니다

LLM 판단은 기록해 두고, 숫자 계산은 코드로 다시 합니다

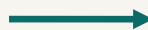
같은 질문에도 LLM 답은 매번 조금씩 달라진다 — 그래서 응답은 저장하고 계산은 결정론으로 둔다

달라질 수 있는 것

LLM 판단

- 국면 판단 · 검토 의견
- IC 토론 · CIO 서술
- llm_cache/ 에 저장

재생 때 저장본을 읽는다



항상 같아야 하는 것

결정론 계산

- 최적화 · 투표 집계
- 지표 · 한도 점검
- 코드로 재계산

다시 돌려도 같은 숫자

에이전트가 숫자를 직접 쓰면 재생 모드에서 재현되지 않습니다

발표는 HTML MVP 의 버튼으로 시작하는 방식을 권장합니다

HTML MVP = 버튼과 결과 화면만 갖춘 최소 웹 화면 · 버튼과 명령은 같은 엔진을 부른다

엔진 한 벌 · 화면 한 벌 — 버튼과 명령은 같은 엔진을 부른다



LLM 대화창은 개발 도구로만 쓰고 발표 실행 경로로는 인정하지 않습니다

대화창에서는 사람이 매번 지시문을 치고 , 맥락에 따라 답이 달라지며 , 기록이 흘러진다

방식	발표 효과	판정
LLM 대화창	친숙하지만 사람이 매번 개입	개발용만
명령 한 줄	재현 · 채점에 강하지만 화면이 딱딱함	필수 (채점 · 대체)
HTML MVP 버튼	진행 · 근거 · 결론을 한 화면에	권장 (발표)

사람 승인 · 긴급 정지를 버튼으로 드러내면 ' 사람은 정해진 곳만 ' 이 화면으로 증명됩니다

MVP 는 Streamlit 이 가장 쉽고 , HTML 은 작은 로컬 서버와 함께 만듭니다

Streamlit = 파이썬 코드만으로 버튼 · 표 · 차트 화면을 만드는 도구 · FastAPI = 파이썬 웹 서버 도구

출발점 · 권장

Streamlit

- 파이썬만으로
- 버튼 · 표 · 차트
- 비전공 팀에 적합

자유도 높음

HTML + FastAPI

- 화면은 HTML
- LLM 호출은 서버
- 키는 서버에만

금지

브라우저 직접 호출

- HTML 에 API 키
- 누구나 키 복사
- 즉시 폐기 대상

MVP 담당 한 명을 정하되 , 화면 코드는 엔진 코드와 다른 폴더에 둡니다

화면은 파일을 읽기만 하고 숫자를 만들지 않습니다

화면이 계산을 하면 버튼 실행과 명령 실행의 결과가 달라진다 — 화면 규칙 세 가지

규칙 1

읽기 전용

- runs/ 파일만 표시
- 계산은 엔진에서
- 편집 기능 없음

규칙 2

실행 번호 표시

- run_id 항상 노출
- 모델 버전 표시
- IPS 버전 표시

규칙 3

버튼 기록

- 누가 · 언제 눌렀나
- 승인 · 정지 사유
- intervention_log

화면의 숫자와 runs/ 파일의 숫자가 다르면 그 시연은 무효입니다

02

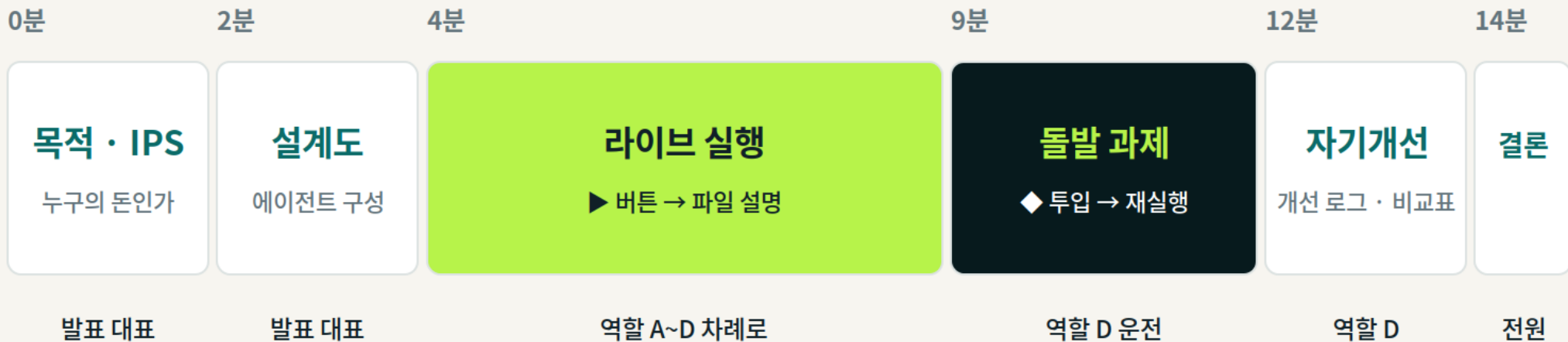
15 분 진행 순서

앞 4 분만 슬라이드로 설명하고 , 나머지는 시스템 화면으로 진행한다

15 분 가운데 11 분은 시스템 화면으로 진행합니다

라이브 실행 5 분이 가장 길다 — MVP 버튼은 역할 D 가 누르고, 각 단계는 해당 역할이 설명한다

발표 15분 — 슬라이드는 앞 4분뿐, 나머지 11분은 시스템 화면



이어서 질의 10분 — 교수가 ▷ 재생 버튼이나 명령 재실행을 요청할 수 있다

MVP 화면은 버튼 · 진행 단계 · 근거 파일 · 결과 요약으로 구성합니다

예시 파일 SDAM_MVP_Example 화면 — 아래쪽에는 실행 비교표와 사람 개입 기록이 이어진다

자율운행 포트폴리오 · 시연 화면 (예시)

run_id: 2026-12-01_02

▶ 라이브 실행

▷ 재생 실행

돌발 A · 위험 한도 강화 ▾

◆ 돌발 과제 투입

↔ 두 실행 비교

✓ 사람 승인

■ 긴급 정지

📄 1쪽 보고서

① 진행 단계

✓ 리서치 브리프

✓ ① 국면 판단

✓ ② CMA · ③ 공분산

✓ ④ 후보 구성 · CRO

✓ ⑤ 검토 · 투표

✓ ⑥ IC 심의

✓ ⑦ CIO 결론

✓ ⑧ 메타 점검

끝난 단계를 누르면 그 단계의 파일이 열린다

② 근거 파일 · ic_report.md

IC 보고서

표결 2 : 2 · 조건부 승인

- IPS 위원: 새 한도 안에서만 승인

- 반대 위원: 주식 축소 폭이 과하다

화면은 runs/ 폴더의 파일을 그대로 읽어 보여 준다

③ 결과 요약

주식		38%
채권		40%
실물		14%
현금		8%

조합법	위험 균형
추적오차 (한도)	1.9% (≤ 4%)
유효 종목 수	10.6
IPS 위반	0건
실행 방식	live · A

실행이 끝나면 IC 권고와 CIO 결정을 1 쪽 보고서로 보여 줍니다

배분 결론은 ⑦ CIO 에이전트의 결정으로 끝난다 — 사람은 결론을 승인하지 않는다

보고서 구성

IC 보고서 · CIO 결정

- ⑥ IC 표결 · 소수의견
- ⑦ CIO 결정 · 사유
- ⑧ 메타 → 다음 주기

인쇄하면 A4 한 장

IC 보고서 · CIO 결정

run_id 2026-12-01_02 · 실행 live · 시나리오 A · 국면 경기 후기 · 모델 llm-x-2026

안전

IPS v2(추적오차 한도 4%) 반영 배분안

⑥ IC 심의 — 권고

표결 2 : 2

IC 권고: 조건부 승인

수익 위원	반대	주식 축소로 기대수익이 낮아진다
위험 위원	찬성	새 한도에 여유가 있다
IPS 위원	찬성	IPS v2 한도를 모두 지킨다
반대 위원	반대	주식 축소 폭이 과하다

소수의견

현금 8%는 과하다 — 다음 주기에 채권으로 옮길 것

⑦ CIO 결정 — 최종

조건부 승인 → 승인으로 확정

조합법: 위험 균형

사유: 표결 동수 — IPS 준수를 우선해 확정, 현금 비중은 다음 주기 재검토

주식 38%	채권 40%	실물 14%	현금 8%
추적오차 1.9% (한도 4%)	유효 종목 수 10.6	IPS 위반 0건	

⑧ 메타 점검 — 다음 주기로

경보: CMA 오차 기준선보다 나쁨 · 큰 수정 → 사람 ⑨ 승인 대기

예시 파일에서 팀이 바꿀 곳은 callEngine() 한 곳입니다

예시는 결과를 파일 안에 넣어 두었다 — 실제 프로젝트에서는 이 함수가 엔진 (서버) 을 부른다

```
// 실제 프로젝트 : 버튼 → 서버의 엔진 실행 → runs/<run_id>/ 파일 내용을 받는다
async function callEngine(mode, scenario) {
  const res = await fetch("/api/run", { method: "POST",
                                     body: JSON.stringify({ mode, scenario }) });
  return res.json();
}
```

화면 코드는 그대로 두고 엔진만 붙이면 됩니다 — 화면에서 숫자를 계산하지 않습니다

API 키는 서버에만 둥니다 — HTML 파일에 넣지 않습니다

라이브 실행 중에는 역할 담당자가 자기 파일을 직접 설명합니다

역할마다 1 분 남짓 — 파일을 열어 핵심 한 가지만 짚는다 (역할 E · F 가 있으면 해당 단계에 끼워 넣는다)

역할	열어 보일 파일	한 줄로 답할 질문
A · 거시 · CMA	regime.json · cma.json	왜 이 국면이고 , 수익 가정은 ?
B · 구성 · CRO	proposals.json · risk	후보 몇 개 중 몇 개가 탈락 ?
C · IC · CIO	ic_report.md · final_alloc	반대 의견과 CIO 의 결론은 ?
D · 메타 · 리서치	meta_report · improvement_log	무엇을 고쳤고 통과했나 ?

자기 역할 파일을 설명하지 못하면 개인 기여 점수에 반영됩니다

IC 보고서에서 CIO 결론까지는 반드시 화면으로 보여 줍니다

이 시스템의 핵심은 심의와 결론을 에이전트가 낸다는 점이다 — 그 과정을 숨기지 않는다

⑥ IC 표결

찬반과 조건

- 위원 4 명 표결
- 승인 · 조건부 · 반려
- IPS 이탈 항목

⑥ 반대 위원

부결된 의견

- 약점 목록
- 지우지 않음
- CIO 가 읽었나

⑦ CIO 결론

최종 배분

- 권고와 같은가
- 다르면 그 이유
- 조합법과 점수

마지막 화면은 IC 권고와 CIO 결정을 담은 1 쪽 보고서입니다

CIO 가 IC 권고를 뒤집은 장면이 있다면 오히려 좋은 시연 소재입니다

버튼마다 같은 일을 하는 명령을 README 첫머리에 적습니다

버튼이 고장 나도 명령으로 같은 실행을 할 수 있어야 한다 — 교수는 채점할 때 이 명령을 쓴다

▶ 라이브 실행 버튼과 같은 명령

```
$ python run_all.py --mode live --asof 2026-11-30
```

▷ 재생 실행 버튼과 같은 명령

```
$ python run_all.py --mode replay --run 2026-11-30_01
```

⇄ 두 실행 비교 버튼과 같은 명령

```
$ python compare_runs.py 2026-11-30_01 2026-12-01_02
```

버튼과 명령이 같은 실행 번호 · 같은 숫자를 내는지 리허설에서 확인합니다

03

돌발 과제 대응

발표 당일 입력 파일로 사건을 주고, 시스템이 스스로 대응하는지 본다

돌발 과제는 발표 당일 입력 파일 하나로 주어집니다

당일 하나를 뽑아 MVP 의  버튼으로 투입하고 , 나머지 둘은 미리 돌린 비교표로 제출한다

과제	교수가 주는 것	팀이 할 일
A · 정책 변경	ips_v2.yaml (위험 한도 강화)	✓ 서명 버튼 → ► 재실행
B · 국면 전환	shock_brief.md (침체 신호)	◆ 투입 → 리서치 → 재심의
C · 모델 교체	새 LLM 이름	설정에서 모델 선택 → ► 재실행

입력 파일 형식은 발표 두 주 전에 공개하고 , 내용은 당일 공개합니다

돌발 A 만 사람이 개입합니다 — B 와 C 는 시스템이 알아서 대응해야 합니다

대응은 코드를 고치지 않고 입력과 설정만 바꿔서 합니다

코드를 고쳐야 대응되는 시스템은 자율 운행이 아니다 — 합격 기준 세 가지로 본다

기준 1

코드 수정 없음

- 입력 · 설정만 교체
- 커밋 기록으로 확인
- 버튼으로 재실행

기준 2

이전 실행 보존

- 새 실행 번호
- 옛 폴더 그대로
- IPS 버전 기록

기준 3

비교표 제시

- 이전 대 이후
- 달라진 판단
- 달라진 이유

모든 후보가 탈락하면 한도를 풀지 말고 현금 비중 유지를 결론으로 냅니다

변화는 두 실행을 나란히 놓은 비교표로 보여 줍니다

⇒ 비교 버튼이 만드는 표의 예시 — 숫자는 설명용이며 실제 값은 팀마다 다르다

항목	이전 실행	돌발 A 이후
주식 비중	44.9%	38.0%
추적오차 (한도)	2.4% (≤ 6%)	1.9% (≤ 4%)
IC 표결	3 : 1 승인	2 : 2 조건부
CIO 조합법	최대 분산	위험 균형
IPS 위반	0 건	0 건

숫자보다 ' 왜 달라졌는지 ' 를 IC 보고서 문장으로 설명합니다

04

준비 · 리허설

발표 당일의 변수를 줄이려면 준비물과 리허설 조건을 미리 정해 둔다

발표 사흘 전까지 시연 꾸러미를 저장소에 올립니다

교수는 이 꾸러미로 발표 뒤 재생 모드를 직접 돌려 본다

꾸러미 1

태그 final

- 코드 고정
- 수정 금지
- 해시 기록

꾸러미 2

llm_cache/

- 재생용 응답
- 실행별 폴더
- 모델 버전

꾸러미 3

녹화 영상

- 전체 실행
- 5 분 이내
- 장애 대비

꾸러미 4

실행 환경

- MVP 실행법
- README
- API 키 제외

API 키를 저장소에 올리면 즉시 폐기하고 새로 발급합니다

리허설은 조건을 바꿔 세 번 합니다

한 번은 정상 조건, 두 번은 장애를 가정한 조건 — 모두 발표 전 주에 끝낸다

리허설 1

시간 재기

- 15 분 통으로
- 역할별 1 분 확인
- 넘치면 줄일 곳

리허설 2

인터넷 끄고

- ▶ 재생 버튼만
- 같은 숫자 확인
- 전환 멘트 연습

리허설 3

다른 노트북

- 새로 clone
- README 대로 설치
- MVP 띄워 실행

리허설 3 을 통과해야 '재현 가능' 이라고 말할 수 있습니다

장애가 나도 멈추지 않도록 대안을 미리 정해 둡니다

장애 자체는 감점하지 않는다 — 숨기거나 멈춰 서는 것을 감점한다

상황

대응

LLM API 오류 · 지연

▶ 재생 버튼으로 바꾸고 전환 사실을 말한다

실행 시간 초과

진행 중 단계만 보이고 저장된 실행으로 넘어간다

MVP 화면 오류

README 의 명령으로 같은 실행을 돌린다

예상과 다른 결과

숨기지 말고 원인 가설과 확인 방법을 말한다

노트북 고장

리허설 3 에서 쓴 다른 노트북으로 바꾼다

모든 것이 실패하면 녹화 영상으로 넘어가되 , 질의에서 재생을 다시 시도합니다

05

평가 · 점검표

최종 발표 10 점과 재현성 점검이 무엇을 보는지 공개한다

최종 발표 10 점은 실행 · 돌발 대응 · 질의로 나눠 봅니다

재현성은 '시스템 완성도 · 재현성' 30 점에서 따로 본다 — 교수가 꾸러미로 재생 모드를 돌려 확인한다

항목	무엇을 보나	배점
라이브 실행	버튼 실행 · 역할별 파일 설명 · 1 쪽 보고서	3
돌발 과제 대응	코드 수정 없음 · 이전 실행 보존 · 비교표	4
질의 응답	설계 이유 · 한계 · 장애 처리	3

MVP 화면의 완성도는 창의 · 독자성 40 점에서 가산합니다

초과수익률이 높다고 시연 점수가 오르지 않습니다

감점되는 시연과 좋은 시연은 정직함에서 갑니다

화면에 보이는 것과 저장소에 남은 것이 같아야 한다

감점

보여 주기용 시연

- 미리 만든 화면 재생
- 화면에서 숫자 계산
- 결과 숫자 손 편집

라이브 실행 점수 없음



좋은 시연

증명하는 시연

- 실패도 화면에
- 파일을 열어 근거
- 전환 사실 공개

질의에서 다시 확인된다

결과를 손으로 고친 흔적이 발견되면 그 실행은 무효입니다

발표 전날에는 여섯 가지를 점검합니다

팀원 한 명이 읽고, 다른 한 명이 화면에서 확인한다

점검 1

태그 final

사흘 전 커밋과 같은가

점검 2

버튼 일곱 개

모두 눌러 봤나

점검 3

재생 모드

인터넷 없이 같은 숫자

점검 4

명령 대체

버튼 없이도 도는가

점검 5

역할별 파일

열 파일과 한 줄 답

점검 6

녹화 · 예비 노트북

바로 꺼낼 수 있나

MVP 글자는 뒷줄에서도 읽히도록 크게 키우고 전체 화면으로 띄웁니다

버튼 하나로 돌리고 ,
근거는 파일에서 보이고 ,
변화는 비교표로 증명한다 .

자율운행 포트폴리오 팀 프로젝트
최종 발표 시연 가이드